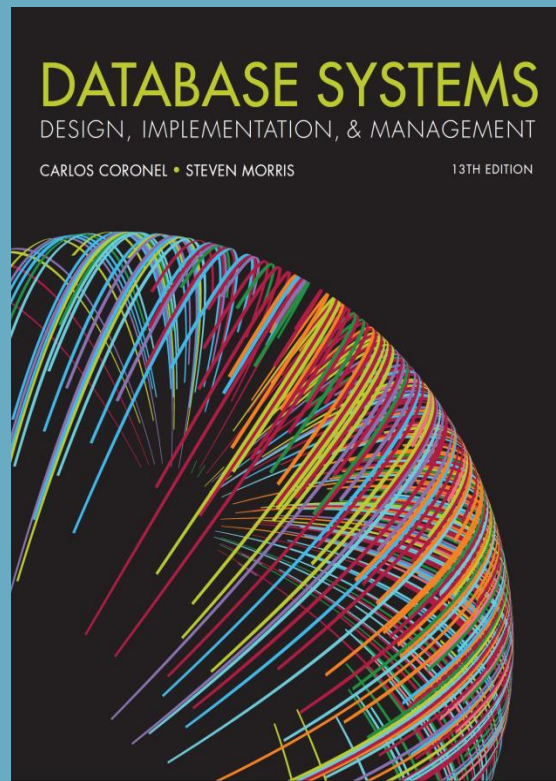




Chapter 12

Distributed Database Management Systems



Learning Objectives

- **After completing this chapter, you will be able to:**
 - Explain the purpose and function of distributed database management systems (DDBMSs)
 - Summarize the advantages and disadvantages of DDBMSs
 - Describe the characteristics and components of DDBMSs
 - Explain how database implementation is affected by different levels of data and process distribution
 - Understand how transactions are managed in a distributed database environment
 - Describe how distributed database design balances performance, scalability and availability
 - Explain the trade-offs of implementing a distributed data system



Some definitions

- **Distributed Database**

- A logically interrelated collection of shared data (and a description of this data), physically distributed over a computer network

- **Distributed DBMS**

- Software system that permits the management of the distributed database and makes the distribution transparent to users



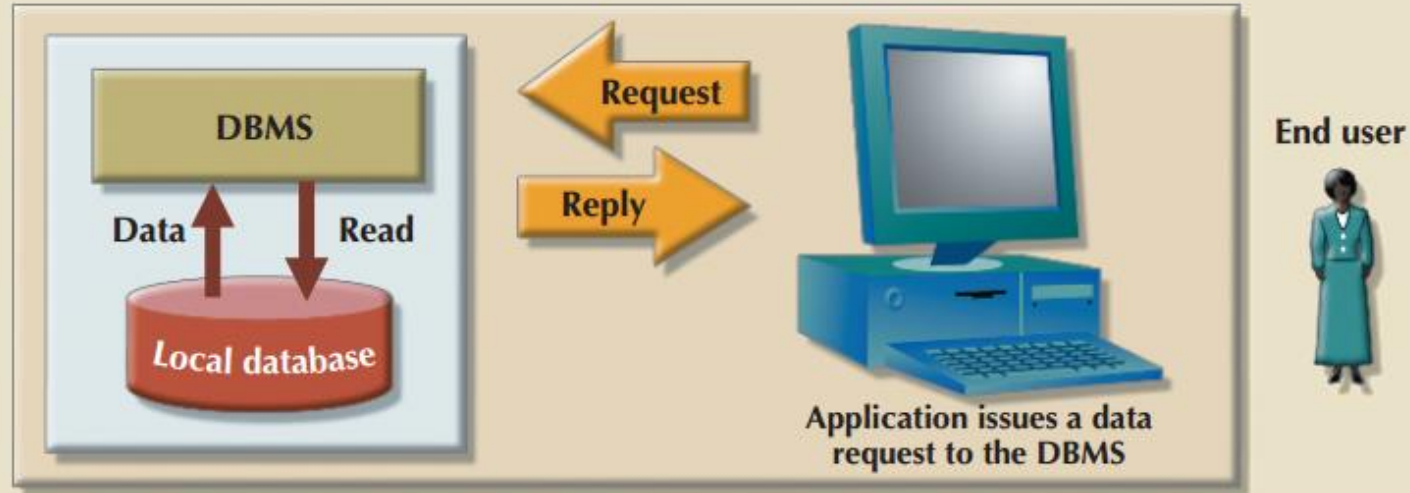
The Evolution of Distributed Database Management Systems

- **A distributed database management system (DDBMS)**
 - Governs storage and processing of logically related data over interconnected computer systems
 - Distributes data and processing functions among several sites
- **Centralized database management system**
 - Data is stored in a single central site
 - Data access provided through dumb terminals



The Evolution of Distributed Database Management Systems

FIGURE 12.1 CENTRALIZED DATABASE MANAGEMENT SYSTEM





The Evolution of Distributed Database Management Systems

- Social and technological changes that affected the nature of systems and the data they use
 - Globalization of business operation
 - Increased market needs for an on-demand transaction style, based on web-based services
 - Rapid social and technological changes fueled by low-cost smart mobile devices
 - Converging data realms in the digital world
 - Advent of social media to reach new customers and markets



The Evolution of Distributed Database Management Systems

- Database requirements in a dynamic business environment
 1. **Rapid ad hoc data access**
 - Became crucial in the quick-response decision-making environment
 2. **Distributed data access**
 - Was needed to support geographically dispersed business units
- **These DB requirements were not met by the centralized DBMS**



The Evolution of Distributed Database Management Systems

- **Factors that influenced the way requirements for a dynamic business environment were addressed**
 - Acceptance of Internet as a platform for business
 - The Web is effectively the repository for distributed data
 - **Mobile wireless revolution** – has created high demand for data access
 - **The accelerated growth of companies using “application as a service”**
 - The service provides remote application services to companies wanting to outsource their application development, maintenance and operations.
 - **The increased focus on mobile business intelligence**
 - As companies use social networks to get closer to customers, the need for on-the-spot decision making increases
 - **Emphasis on Big Data analytics** – The era of mobile communications unraveled an avalanche of data from many sources and of many types



The Evolution of Distributed Database Management Systems

- **Potential centralized DBMS problems**
 - **Performance degradation** because of a growing number of remote locations over greater distances
 - **High costs** associated with maintaining and operating large central (mainframe) data-base systems and physical infrastructure
 - **Reliability problems** created by dependence on a central site (single point of failure syndrome) and the need for data replication
 - **Scalability problems** associated with the physical limits imposed by a single location, such as physical space, temperature conditioning, and power consumption
 - **Organizational rigidity** imposed by the database, which means it might not support the flexibility and agility required by modern global organizations



DDBMS Advantages and Disadvantages

Table 12.1: Distributed DBMS Advantages and Disadvantages

Advantages	Disadvantages
Data is located near the site of greatest demand. The data in a distributed database system is dispersed to match business requirements.	Complexity of management and control. Applications must recognize data location and be able to stitch together data from various sites. Database administrators must have the ability to coordinate database activities to prevent database degradation due to data anomalies.
Faster data access. End users often work with only the nearest stored subset of the data.	Technological difficulty. Data integrity, transaction management, concurrency control, security, backup, recovery, and query optimization must all be addressed and resolved.
Faster data processing. A distributed database system spreads out the system's workload by processing data at several sites.	Security. Probability of security lapses increases when data is located at multiple sites. Responsibility of data management will be shared by different people at several sites.
Growth facilitation. New sites can be added to the network without affecting the operations of other sites.	Lack of standards. No standard communication protocols at the database level. Different database vendors employ different and often incompatible techniques to manage the distribution of data and processing in a DDBMS environment.
Improved communications. Because local sites are smaller and located closer to customers, local sites foster better communication among departments and between customers and company staff.	Increased storage and infrastructure requirements. Multiple copies of data are required at different sites, thus requiring additional storage space.
Reduced operating costs. It's more cost-effective to add nodes to a network than to update a mainframe system. Development work is done more cheaply and quickly on low-cost PC and laptops than on mainframes.	Increased training cost. Training costs are generally higher in a distributed model than they would be in a centralized model, sometimes even to the extent of offsetting operational and hardware savings.
User-friendly interface. Client devices are usually equipped with an easy-to-use graphical user interface (GUI). The GUI simplifies training and use for end users.	Higher costs. Distributed databases require duplicated infrastructure to operate, such as physical location, environment, personnel, software, and licensing.
Less danger of a single-point failure. When one of the computers fails, the workload is picked up by other workstations. Data is also distributed at multiple sites.	
Processor independence. The end user can access any available copy of the data, and an end user's request is processed by any processor at the data location.	



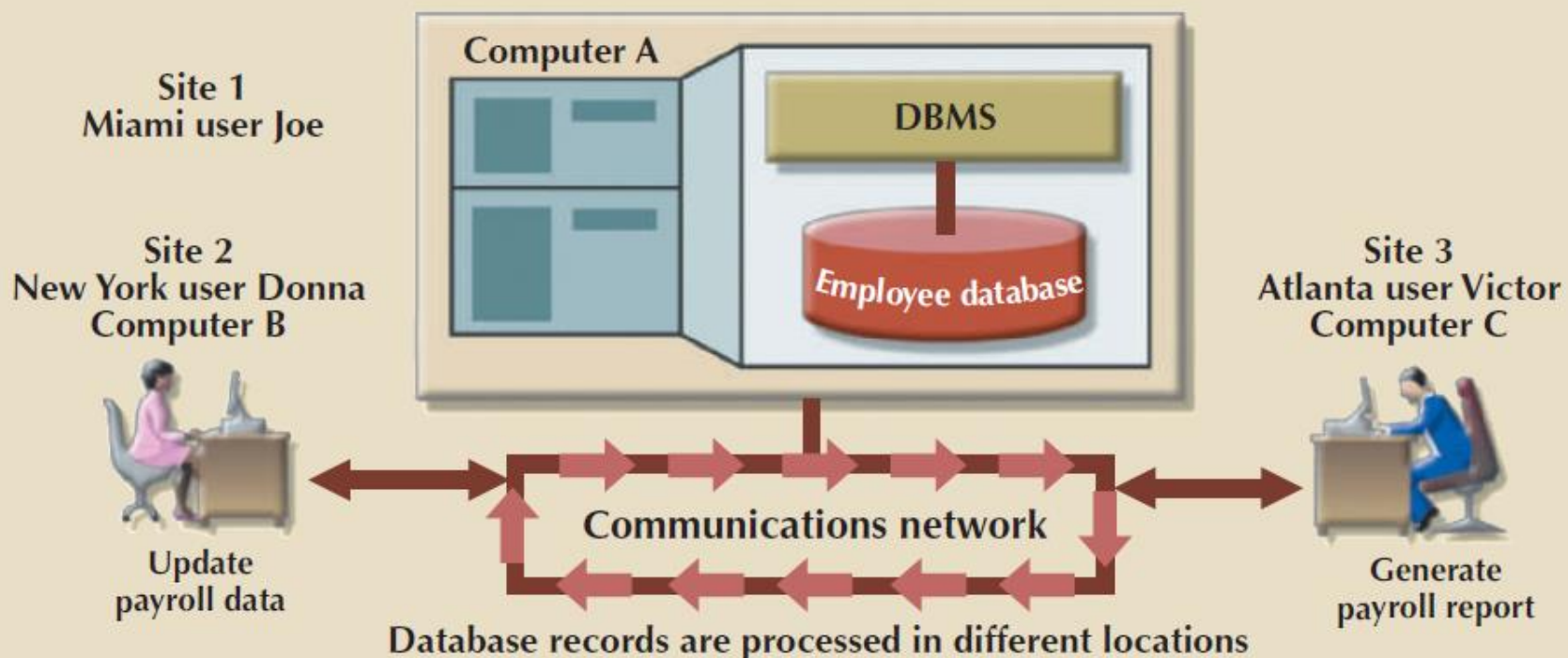
Distributed Processing and Distributed Databases

- **Distributed processing**: database's logical processing is shared among two or more physically independent sites via network
- **Distributed database**: stores logically related database over two or more physically independent sites via a computer network
 - Database is composed of several parts known as **database fragments**
 - Database fragments are located at different sites



Distributed Processing

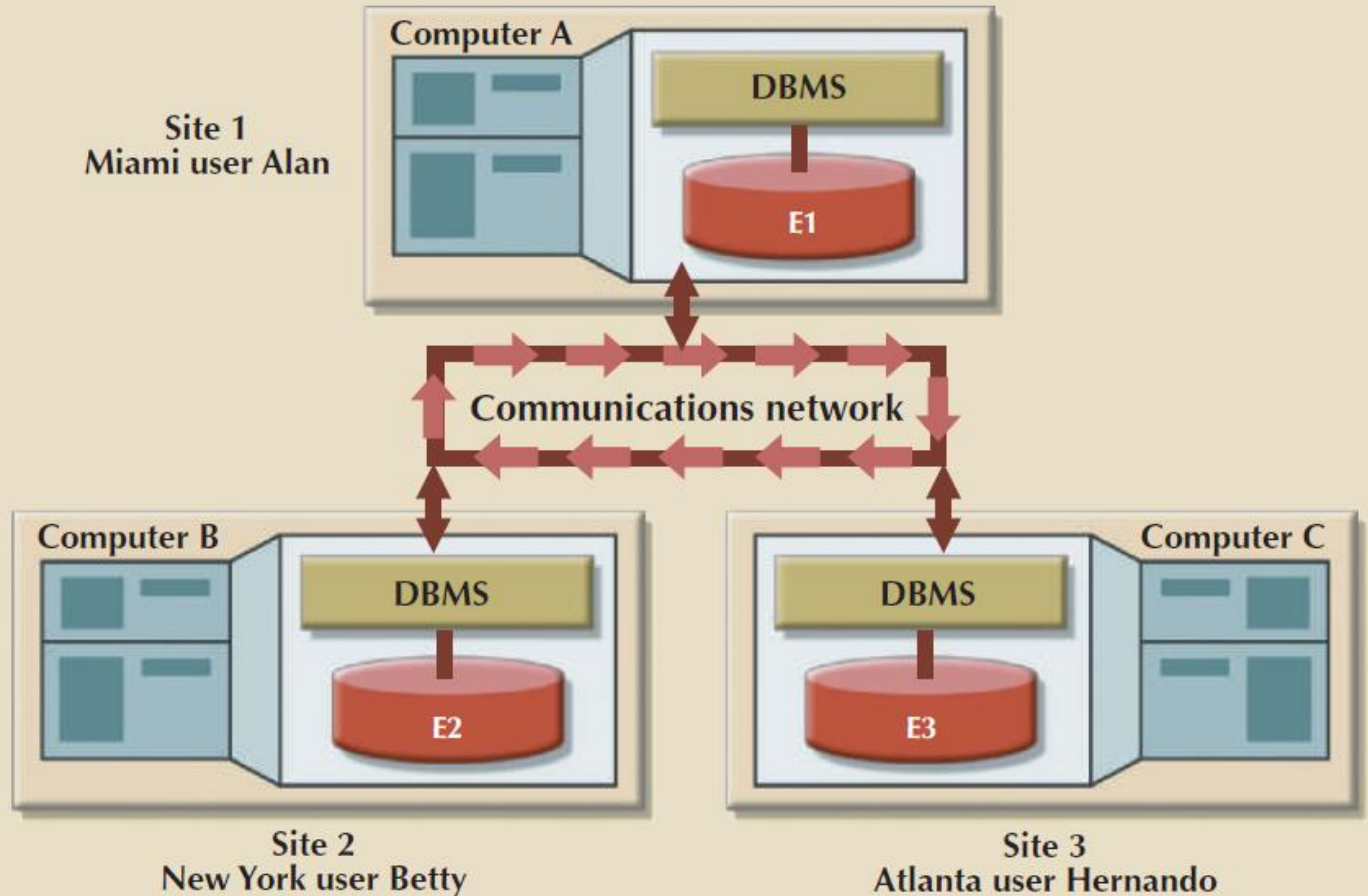
FIGURE 12.2 DISTRIBUTED PROCESSING ENVIRONMENT





Distributed Database

FIGURE 12.3 DISTRIBUTED DATABASE ENVIRONMENT





Distributed processing vs distributed database

- **Keep the following points in mind:**
 - Distributed processing does not require a distributed database, but a distributed database requires distributed processing
 - Each database fragment is managed by its own local database process
 - Distributed processing may be based on a single database located on a single computer.
 - For the management of distributed data to occur, copies or parts of the database processing functions must be distributed to all data storage sites.
 - Both distributed processing and distributed databases require a network to connect all components



Characteristics of Distributed Management Systems

- A DBMS must have several functions to be classified as distributed
 - **Application interface** to interact with the end user, application programs and other DBMSs within the distributed database
 - **Validation** to analyse data requests for syntax correctness
 - **Transformation** to decompose complex requests into atomic data request components
 - **Query optimization** to find the best access strategy. (Which database fragments must be accessed by the query and how must data updates, if any, be synchronized?)
 - **Mapping** to determine the data location of local and remote fragments
 - **I/O interface** to read or write data from or to permanent local storage



Characteristics of Distributed Management Systems

- A DBMS must have several functions to be classified as distributed
 - **Formatting** to prepare the data for presentation to the end user or to an application program
 - **Security** to provide data privacy at both local and remote databases
 - **Backup and recovery** to ensure the availability and recoverability of the database in case of a failure
 - **DB administration** features for the database administrator
 - **Concurrency control** to manage simultaneous data access and to ensure data consistency across database fragments in the DDBMS
 - **Transaction management** to ensure that the data moves from one consistent state to another



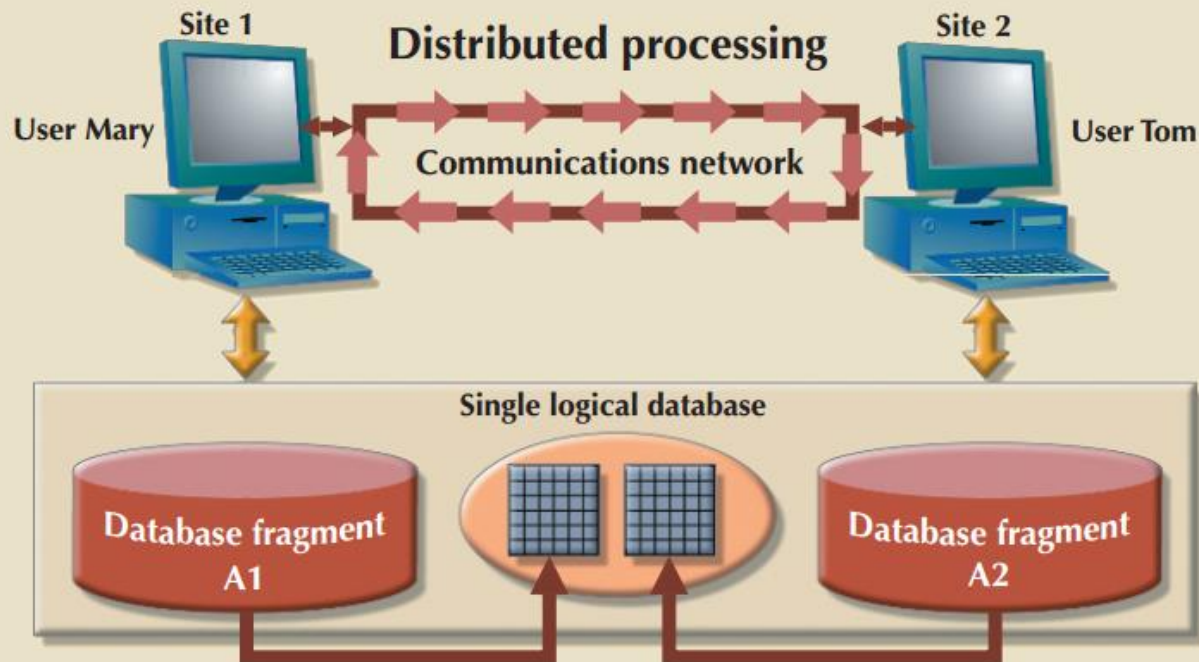
Characteristics of Distributed Management Systems

- Apart from the functions on the previous slide, a fully distributed database management system performs functions of a centralized DBMS, as follows:
 - Receive the request of an application or end user
 - Validate, analyze, and decompose the request
 - Map request's logical-to-physical data components
 - Decompose request into several I/O operations
 - Search, locate, read and validate data
 - Ensure database consistency, security, and integrity
 - Validate data for conditions specified by request
 - Present data in required format
 - Handle all necessary functions transparently to user



Characteristics of Distributed Management Systems

FIGURE 12.4 A FULLY DISTRIBUTED DATABASE MANAGEMENT SYSTEM





DDBMS Components

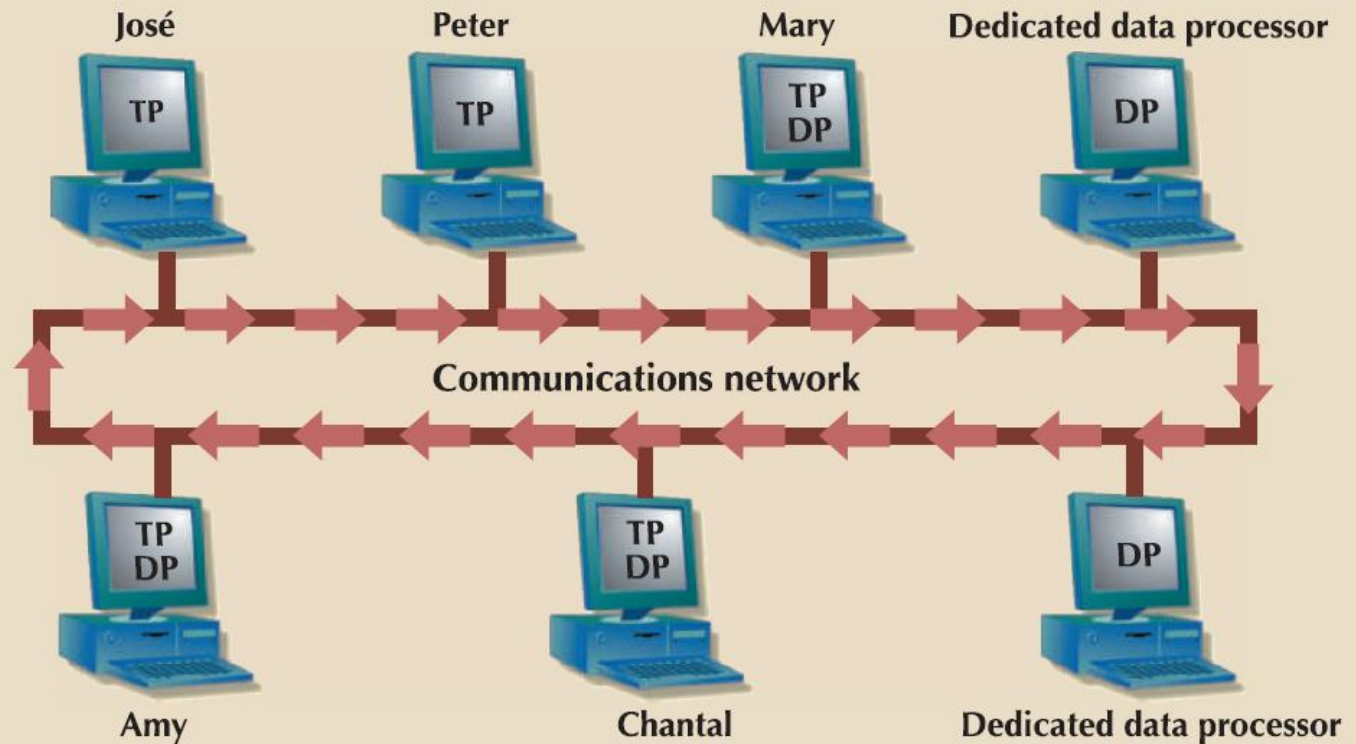
- The DDBMS must include at least the following components:
 - Computer workstations or remote devices that form the network system
 - Network hardware and software components residing in workstations
 - Communications media that carry the data from one node to another
 - Transaction processor (TP) – software component found in each computer that receives and processes application's remote and local data requests
 - Known as transaction manager (TM) or application processor (AP)
 - Data processor (DP) or data manager (DM) – software component found on each computer or device that stores and retrieves data located at the site



DDBMS Components

- The figure below illustrates the placement of the components and the interaction among them

FIGURE 12.5 DISTRIBUTED DATABASE SYSTEM COMPONENTS



Note: Each TP can access data on any DP, and each DP handles all requests for local data from any TP.



DDBMS Components

- The communication among TPs and DPs shown in Figure 12.5 is made possible through **a specific set of rules, or protocols**
- The protocols determine how the distributed database system will:
 - Interface with the network to transport data and commands between data processors (DPs) and transaction processors (TPs)
 - Synchronize all data received from DPs (TP side) and route retrieved data to the appropriate TPs (DP side)
 - Ensure common database functions in a distributed system. Such functions include security, concurrency control, backup, and recovery.



Levels of Data and Process Distribution

- Current database systems can be classified on the basis of how **process distribution** and **data distribution** are supported
- Table 12.2 uses a simple matrix to classify database systems according to data and process distribution

TABLE 12.2

DATABASE SYSTEMS: LEVELS OF DATA AND PROCESS DISTRIBUTION

ADVANTAGES	SINGLE-SITE DATA	MULTIPLE-SITE DATA
Single-site process	Host DBMS	Not applicable (Requires multiple processes)
Multiple-site process	File server Client/server DBMS (LAN DBMS)	Fully distributed Client/server DDBMS



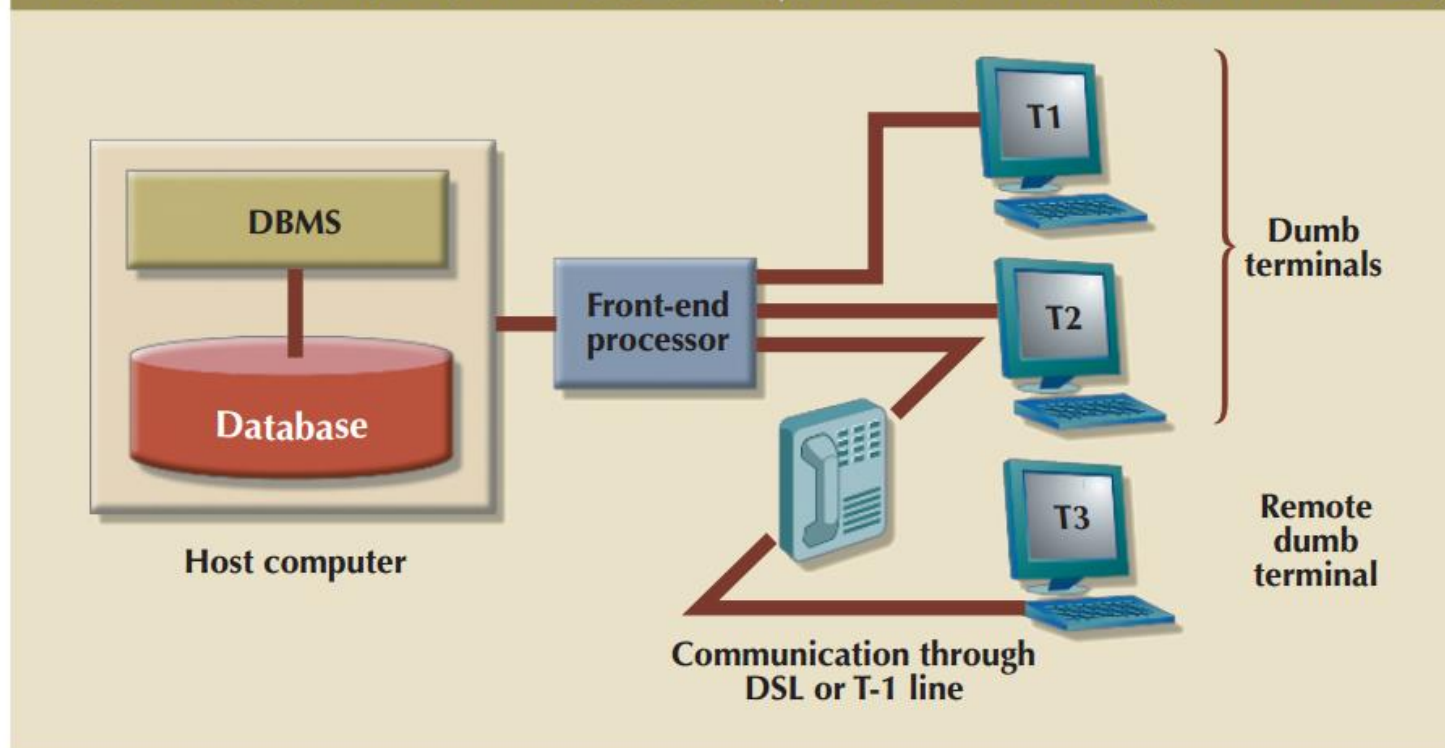
Single-Site Processing, Single-Site Data (SPSD)

- All processing is done on a single host computer and all data are stored on the host computer's local disk system
- Such a scenario is typical of most mainframe and midrange server computer DBMSs
- The DBMS is located on the host computer, which is accessed by dumb terminals connected to it



Single-Site Processing, Single-Site Data (SPSD)

FIGURE 12.6 SINGLE-SITE PROCESSING, SINGLE-SITE DATA (CENTRALIZED)





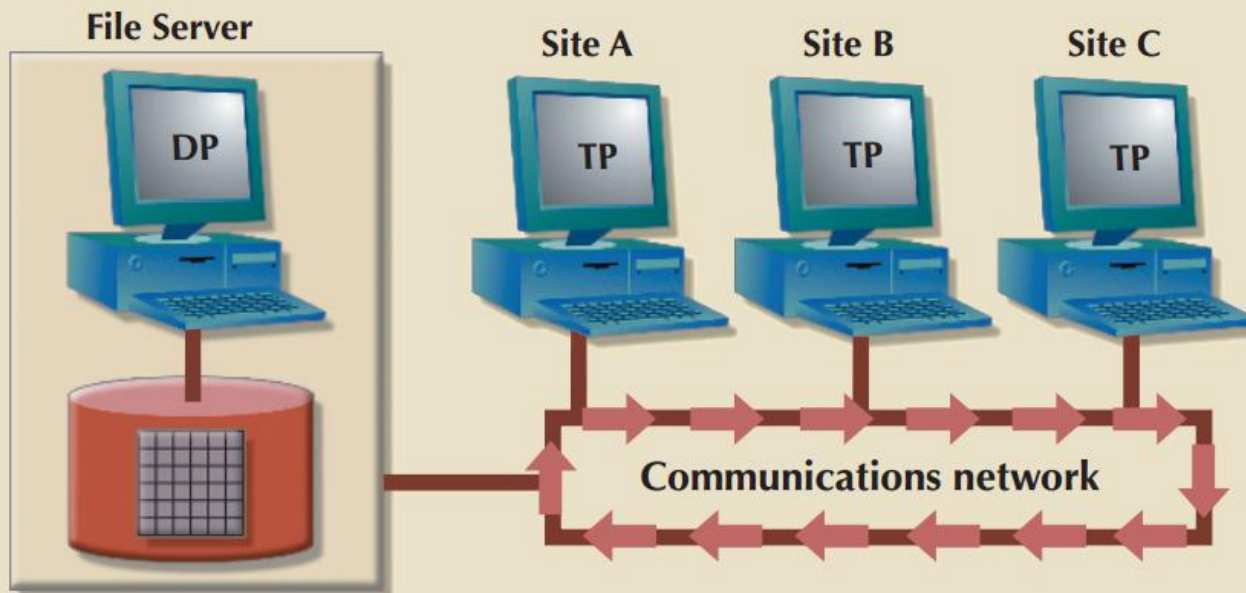
Multiple-Site Processing, Single-Site Data (MPSD)

- Multiple processes run on different computers sharing a single data repository
- Typically requires network file server running conventional applications – accessed through LAN
- **Note the following for MPSD implemented using network file server:**
 - The TP on each workstation acts only as a redirector to route all network data requests to the file server
 - The end user sees the file server as just another hard disk
 - The end user must make a direct reference to the file server in order to access remote data
 - All data selection, search and update functions take place at the workstation, thus requiring that entire files travel through the network for processing at the workstation



Multiple-Site Processing, Single-Site Data (MPSD)

FIGURE 12.7 MULTIPLE-SITE PROCESSING, SINGLE-SITE DATA





Multiple-Site Processing, Single-Site Data (MPSD)

- A variation of the multiple-site processing, single-site data approach is known as **client/server architecture**.
- **Client/server architecture** is similar to that of the **network file server** except that all database processing is done at the server site
 - **Consequently reduces network traffic**
- Client/server systems support distributed multiple-site processing
- Client/server architecture is capable of supporting data at multiple sites
 - **The network file server approach requires the database to be located at a single site**



Multiple-Site Processing, Multiple-Site Data (MPMD)

- Fully distributed database management system
 - Support multiple data processors and transaction processors at multiple sites
- Classifications (based on the level of support for various types of databases):
 - **Homogeneous**: integrate multiple instances of same DBMS over a network
 - **Heterogeneous**: integrate different types of DBMSs over a network but all support the same data model
 - **Fully heterogeneous**: support different DBMSs, each supporting different data model running under different computer systems



Multiple-Site Processing, Multiple-Site Data (MPMD)

- **Restrictions of DDBMS**

- Remote access is provided on a read-only basis and does not support write privileges
- Restrictions are placed on the number of remote tables that may be accessed in a single transaction
- Restrictions are placed on the number of distinct databases that may be accessed
- Restrictions are placed on the database model that may be accessed



Distributed Database Transparency Features

- Minimum desirable DDBMS transparency features
 - **Distribution transparency** – allows a distributed database to be treated as a single logical database
 - **Transaction transparency** – allows a transaction to update data at more than one network site
 - **Failure transparency** – ensures that the system continues to operate in the event of a node failure
 - **Performance transparency** – allows the system to perform as if it were a centralized DBMS
 - **Heterogeneity transparency** – allows the integration of several different local DBMSs (**relational, network and hierarchical**) under a common or global schema



Distribution Transparency

- Allows management of physically dispersed database as if it were a centralized database
- Three levels of distribution transparency are recognized:
 - **Fragmentation transparency**: end user or programmer does not need to know that a database is partitioned
 - **Location transparency**: exists when the end user must specify the database fragment names but does not need to specify where those fragments are located
 - **Local mapping transparency**: exists when the end user or programmer must specify both the fragment names and their locations



Distribution Transparency

- Transparency features are summarized in the table below

TABLE 12.4

SUMMARY OF TRANSPARENCY FEATURES

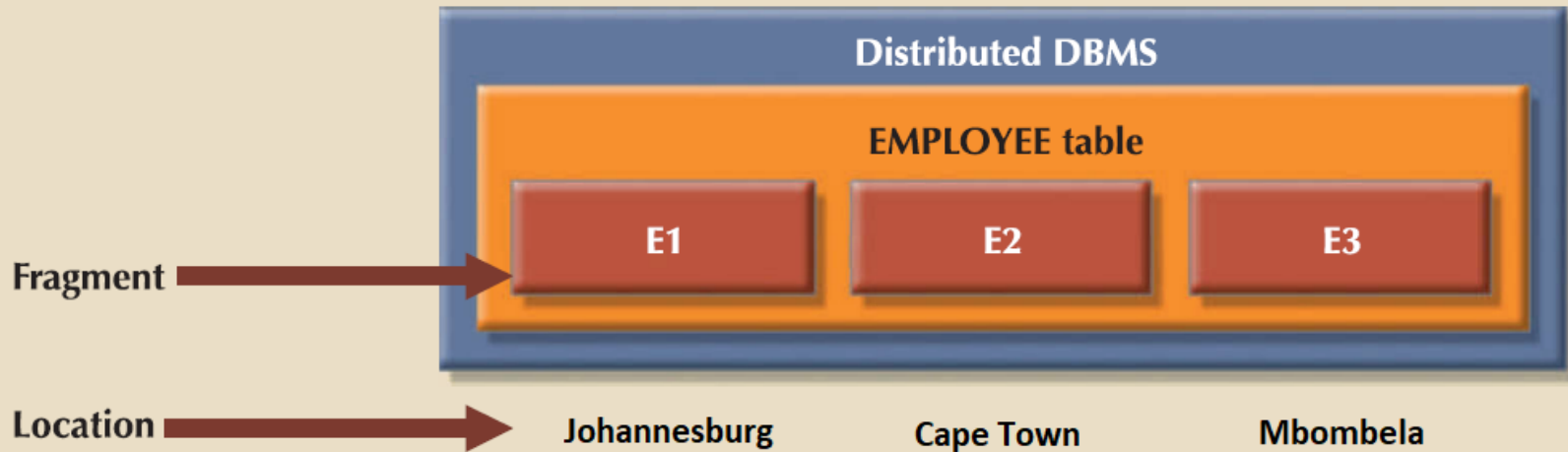
IF THE SQL STATEMENT REQUIRES:

FRAGMENT NAME?	LOCATION NAME?	THEN THE DBMS SUPPORTS	LEVEL OF DISTRIBUTION TRANSPARENCY
Yes	Yes	Local mapping transparency	Low
Yes	No	Location transparency	Medium
No	No	Fragmentation transparency	High



Distribution Transparency Example

FIGURE 12.8 FRAGMENT LOCATIONS



- Depending on the level of distribution transparency support, three query cases may be examined



Distribution Transparency Example

- Case 1. The Database Supports Fragmentation Transparency

```
SELECT *  
FROM EMPLOYEE  
WHERE EMP_DOB < '01-JAN-1979';
```



Distribution Transparency Example

- Case 2: The Database Supports Location Transparency

```
SELECT * FROM E1
WHERE EMP_DOB < '01-JAN-1979'
UNION
SELECT * FROM E2
WHERE EMP_DOB < '01-JAN-1979'
UNION
SELECT * FROM E3
WHERE EMP_DOB < '01-JAN-1979'
```



Distribution Transparency Example

- Case 3: The Database Supports Local Mapping Transparency

```
SELECT *          FROM E1 NODE JOHANNESBURG
WHERE EMP_DOB < '01-JAN-1979';

UNION

SELECT *          FROM E2 NODE CAPE TOWN
WHERE EMP_DOB < '01-JAN-1979';

UNION

SELECT *          FROM E3 NODE MBOMBELA
WHERE EMP_DOB < '01-JAN-1979';
```



Transaction Transparency

- DDBMS property that ensures database transactions maintain the **distributed database's integrity and consistency**
- A DDBMS database transaction can update data stored in many different computers connected in a network
- **Transaction transparency ensures that a transaction completes only when all database sites involved complete their part**

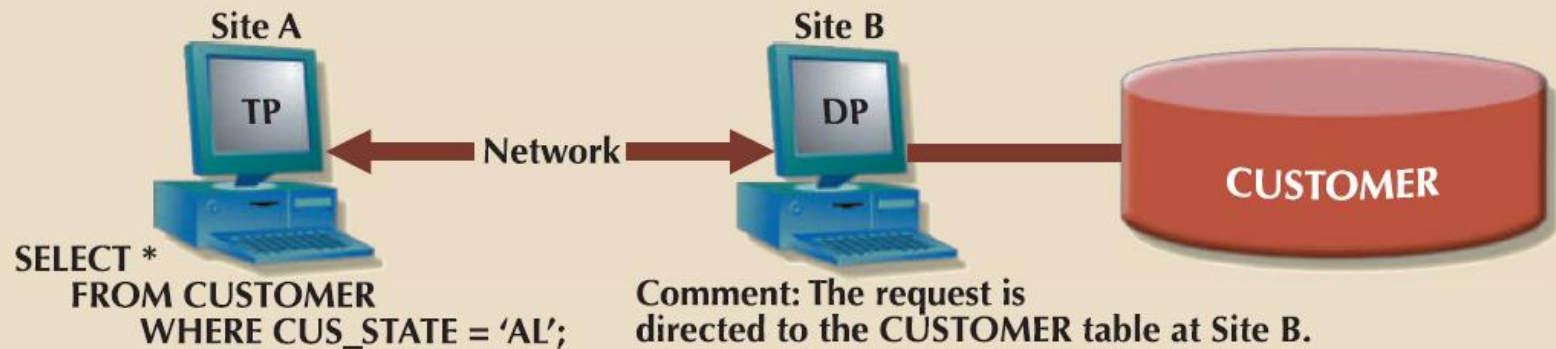


Distributed Requests and Distributed Transactions

- **Remote request**

- Let a single SQL statement access the data that are to be processed by a single remote database processor
- In other words, the SQL statement (or request) references data at only one remote site

FIGURE 12.9 A REMOTE REQUEST

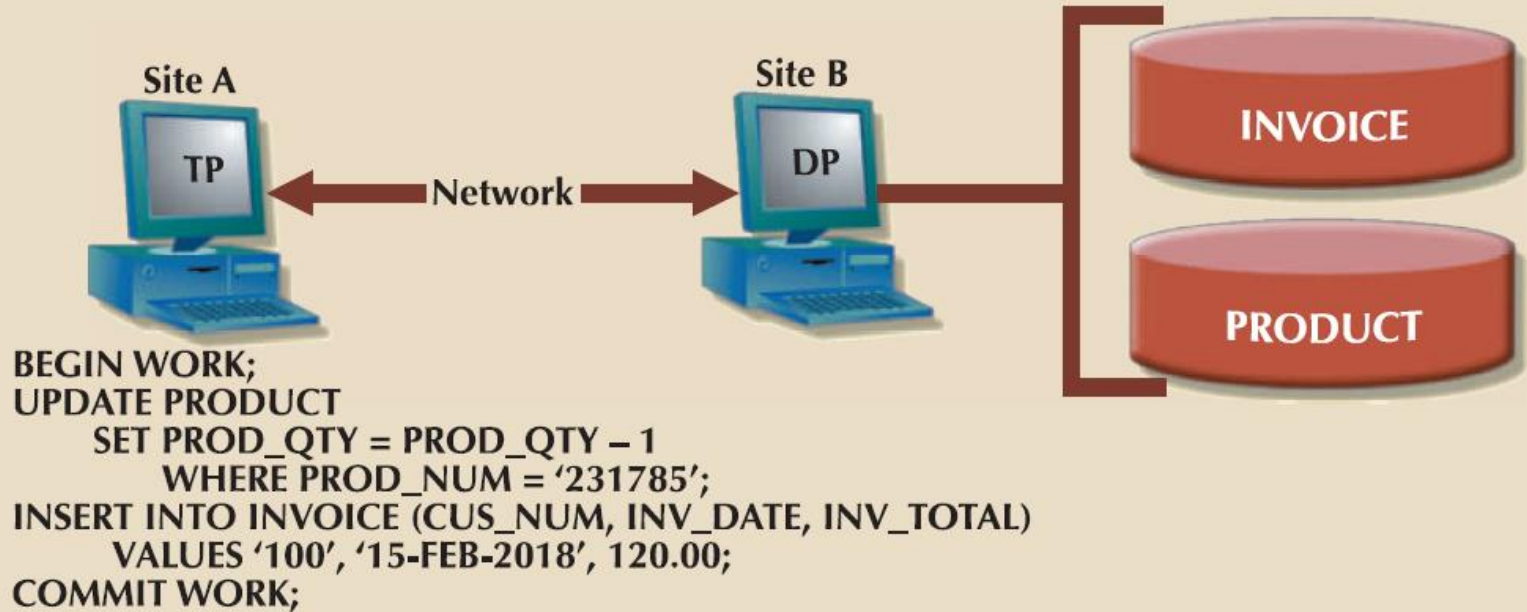




Distributed Requests and Distributed Transactions

- **Remote transaction**
 - Accesses data at a single remote site
 - Is composed of several requests

FIGURE 12.10 A REMOTE TRANSACTION



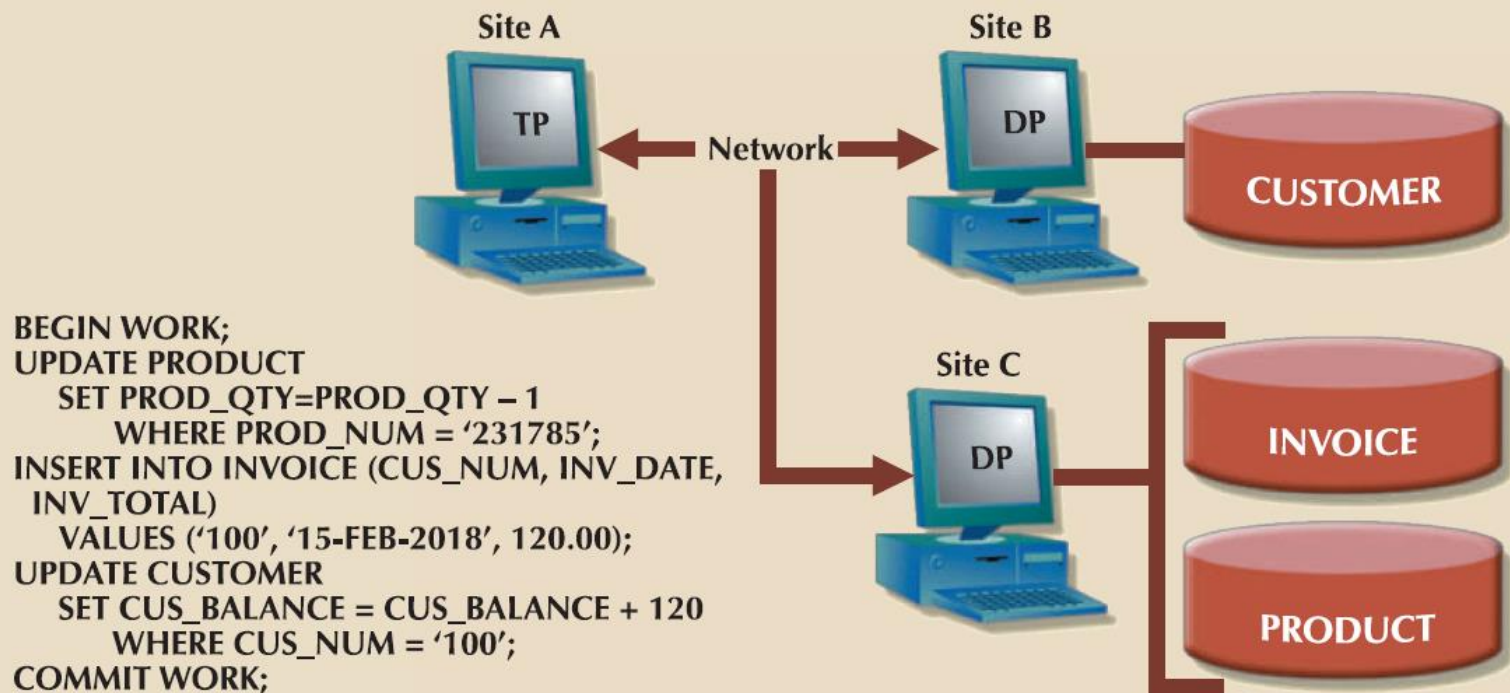


Distributed Requests and Distributed Transactions

- **Distributed transaction**

- Allows a transaction to reference several different local or remote DP sites

FIGURE 12.11 A DISTRIBUTED TRANSACTION





Distributed Requests and Distributed Transactions

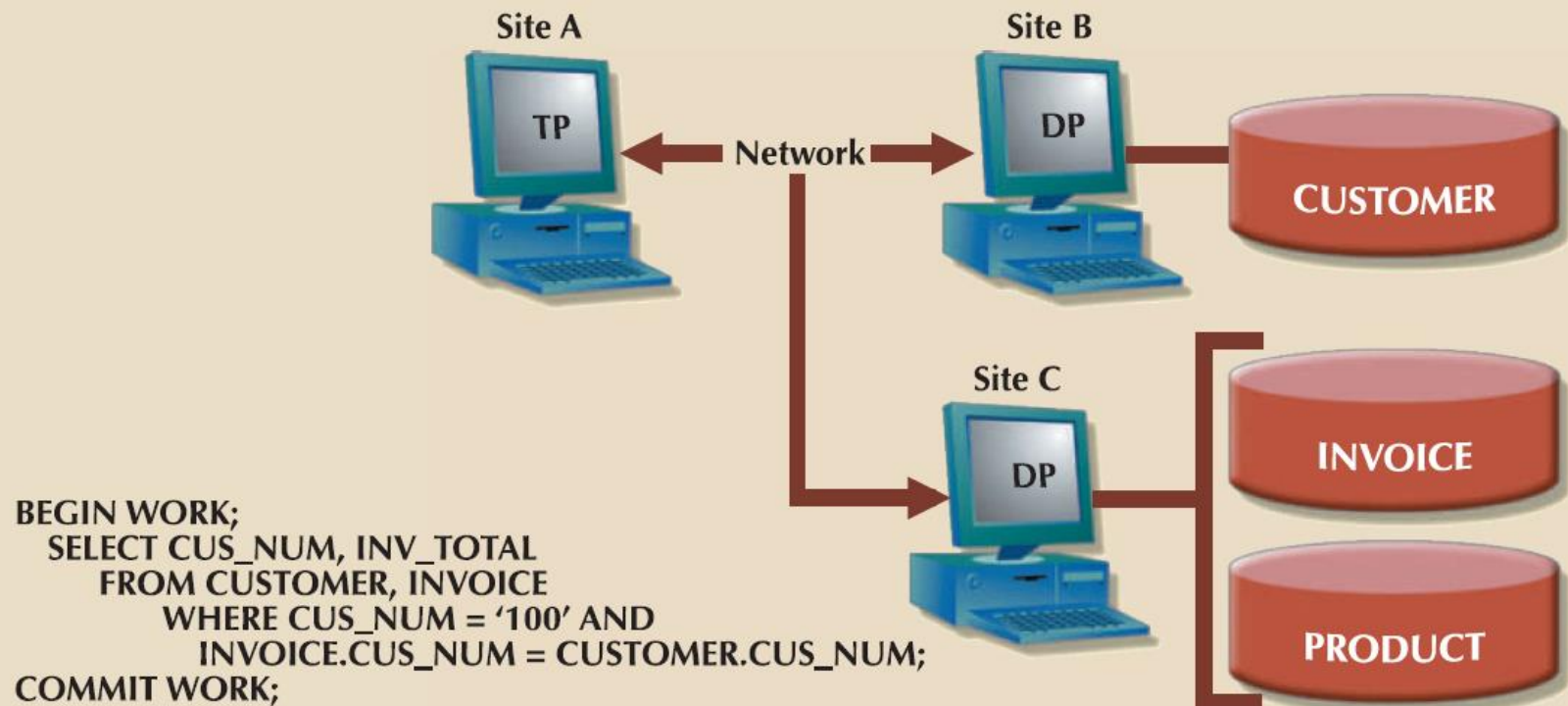
- **Distributed request**

- Let a single SQL statement reference data located at several different local or remote DP sites
- Because each request (SQL statement) can access data from more than one local or remote DP site, **a transaction can access several sites**
- The ability to execute a distributed request provides fully distributed database processing capabilities because of the ability to:
 - **Partition a database table into several fragments**
 - **Reference one or more of those fragments with only one request, i.e., there is fragmentation transparency**



Distributed Requests and Distributed Transactions

FIGURE 12.12 A DISTRIBUTED REQUEST

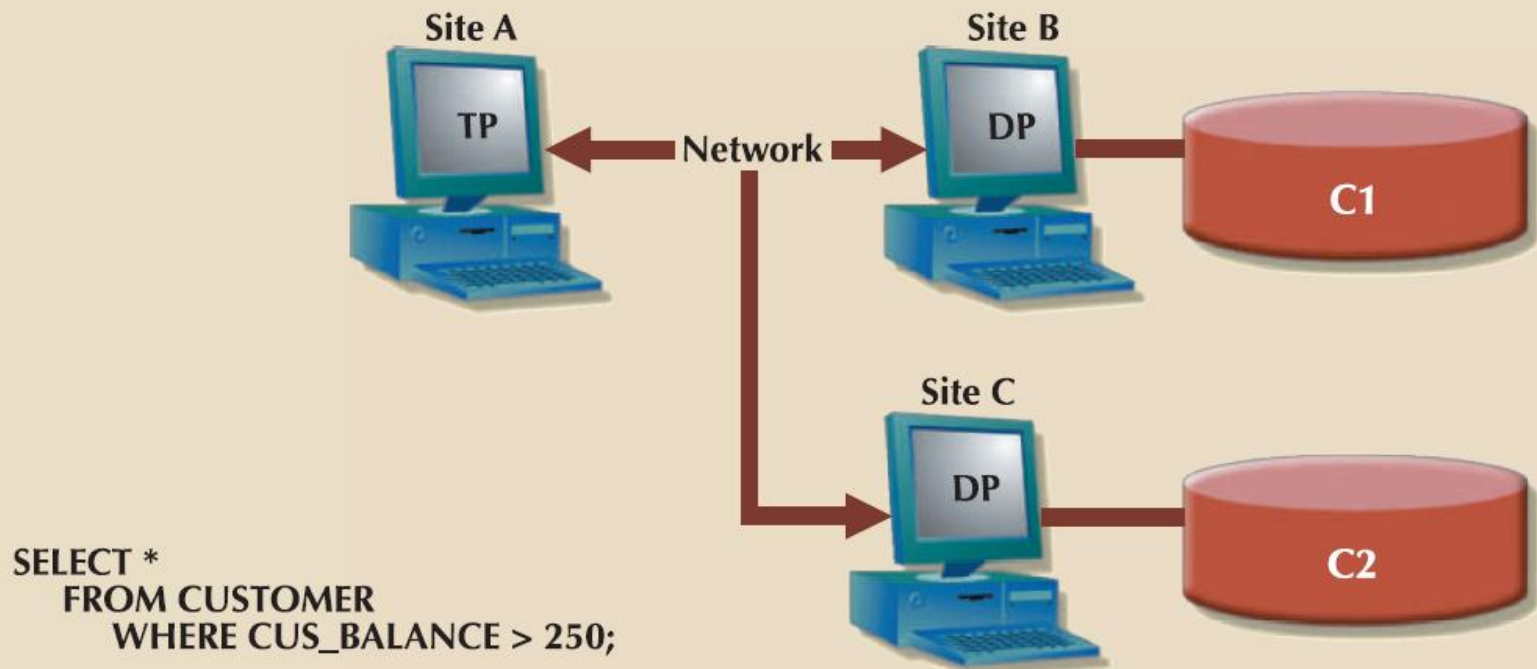




Distributed Requests and Distributed Transactions

- **Distributed request** feature also allows a single request to reference a physically partitioned table

FIGURE 12.13 ANOTHER DISTRIBUTED REQUEST





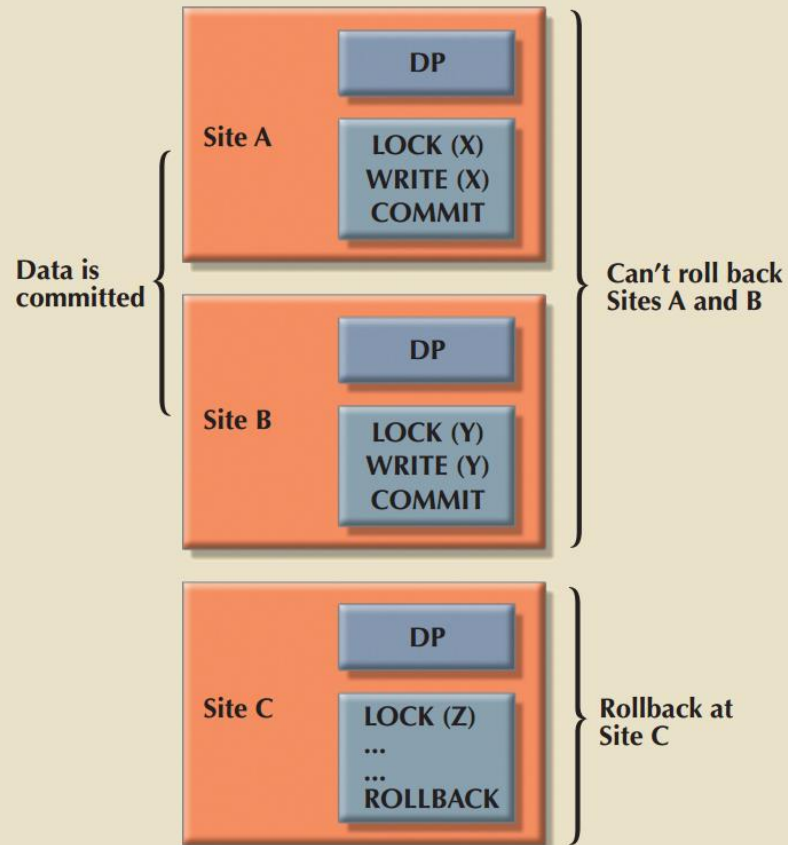
Distributed Concurrency Control

- Concurrency control is especially important in distributed databases environment
 - Multi-site, multiple-process operations are more likely to create **inconsistencies** and **deadlocked transactions**
 - TP component of a DDBMS must ensure that all parts of the transaction are completed at all sites before a final COMMIT is issued to record the transaction



Distributed Concurrency Control

FIGURE 12.14 THE EFFECT OF A PREMATURE COMMIT





Two-Phase Commit (2PC) Protocol

- Guarantees that if a portion of a transaction operation cannot be committed, all changes made at the other sites are undone
 - Maintains a consistent database state
- Requires each DP's transaction log entry be written before database fragment is updated
 - **DO-UNDO-REDO protocol**: rolls transactions back and forward with the help of the system's transaction log entries
 - **Write-ahead protocol**: forces the log entry to be written to permanent storage before actual operation takes place
- **Two-phase commit protocol** defines operations between two types of nodes: the **coordinator** and **subordinates** or cohorts
- **Phases of implementation**
 - Phase 1: Preparation
 - Phase 2: Final COMMIT



Performance and Failure Transparency

- Performance transparency allows a DDBMS to perform as if it were a centralized database
 - i.e., no performance degradation should be incurred due to data distribution
- Failure transparency ensures the system operates in case of network failure
- Objective of query optimization is to minimize total cost associated with the execution of a request.
- The costs associated with a request are a function of the:
 - Access time (I/O) cost involved in accessing data from multiple remote sites
 - Communication costs associated with data transmission
 - CPU time cost associated with the processing overhead



Performance and Failure Transparency

- Considerations for resolving data requests in a distributed data environment:
 - **Data distribution** – the DDBMS must decide which fragments to access (distribution transparency)
 - **Data replication** – the DDBMS must ensure that all copies of the data are consistent
 - **Replica transparency**: DDBMS's ability to hide multiple copies of data from the user
 - **Network and node availability**
 - **Network latency**: delay imposed by the amount of time required for a data packet to make a round trip
 - **Network partitioning**: delay imposed when nodes become suddenly unavailable due to a network failure



Distributed Database Design

- The design of a distributed database introduces three new issues:
 - **Data fragmentation**
 - How to partition database into fragments
 - **Data replication**
 - Which fragments to replicate
 - **Data allocation**
 - Where to locate those fragments and replicas



Data Fragmentation

- Breaks a single object into two or more segments
 - Information is stored in distributed data catalog (DDC), from which it is accessed by the TP to process user requests
- Strategies based at the table level:
 - **Horizontal fragmentation**: division of a relation into subsets (fragments) of tuples (rows)
 - **Vertical fragmentation**: division of a relation into attribute (column) subsets
 - **Mixed fragmentation**: combination of horizontal and vertical strategies



Data Replication

- Storage of data copies at multiple sites served by a computer network
 - Enhance data availability and response time
 - Reduce communication and total query costs
- Replicated data are subject to the **mutual consistency rule**
 - Requires all copies of data fragments be identical
- **Styles of replication:**
 - Push replication focuses on maintaining data consistency
 - Pull replication focuses on maintaining data availability and allows for temporary data inconsistencies



Data Replication

- **Data replication scenarios:**
 - **Fully replicated database:**
 - Stores multiple copies of each database fragment at multiple sites
 - **Partially replicated database:**
 - Stores multiple copies of some database fragments at multiple sites
 - **Unreplicated database:**
 - Stores each database fragment at a single site



Data Allocation

- Describes the process of deciding where to locate data
- **Data allocation strategies:**
 - **Centralized data allocation:** entire database stored at one site
 - **Partitioned data allocation:** database is divided into two or more disjointed fragments and stored at two or more sites
 - **Replicated data allocation:** copies of one or more database fragments are stored at several sites



The CAP Theorem

- Dr Eric Brewer stated that in a highly distributed data system, there are three commonly desirable characteristics: **consistency, availability and partition tolerance**
 - The acronym CAP stands for these properties
- However, it is impossible for a system to provide all three properties at the same time
- **Consistency** – All nodes should see the same data at the same time, which means the replicas should be immediately updated.
- **Availability** – No received request is ever lost
- **Partition tolerance** – The system continues to operate even in the event of a node failure. The system will fail only if all nodes fail.



The CAP Theorem

- To achieve higher availability, some companies tend to forfeit the consistency and isolation components of the ACID properties
- This trade-off between consistency and availability has generated a new type of distributed data system in which data is **basically available, soft state, eventually consistent (BASE)**
 - BASE refers to a data consistent model in which data changes are not immediate but propagate slowly through the system until all replicas are consistent



The CAP Theorem

A spectrum of consistency ranging from **highly consistent (ACID)** to the **eventually consistent (BASE)** of various distributed database types

TABLE 12.8

DISTRIBUTED DATABASE SPECTRUM

DBMS TYPE	CONSISTENCY	AVAILABILITY	PARTITION TOLERANCE	TRANSACTION MODEL	TRADE-OFF
Centralized DBMS	High	High	N/A	ACID	No distributed data processing
Relational DBMS	High	Relaxed	High	ACID (2PC)	Sacrifices availability to ensure consistency and isolation
NoSQL DDBMS	Relaxed	High	High	BASE	Sacrifices consistency to ensure availability
NewSQL DDBMS	High	High	Relaxed	ACID	Sacrifices partition tolerance to ensure transaction consistency and availability



C. J. Date's 12 Commandments for Distributed Databases

Table 12.9	C. J. Date 's 12 Commandments for Distributed Databases	
Rule Number	Rule Name	Rule Explanation
1	Local-site independence	Each local site can act as an independent, autonomous, centralized DBMS. Each site is responsible for security, concurrency control, backup, and recovery.
2	Central-site independence	Each local site can act as an independent, autonomous, centralized DBMS. Each site is responsible for security, concurrency control, backup, and recovery.
3	Failure independence	The system is not affected by node failures. The system is in continuous operation even in the case of a node failure or an expansion of the network.
4	Failure Independence	The user does not need to know the location of data to retrieve that data.
5	Fragmentation transparency	Data fragmentation is transparent to the user, who sees only one logical database. The user does not need to know the name of the database fragments to retrieve them.
6	Replication transparency	The user sees only one logical database. The DDBMS transparently selects the database fragment to access. To the user, the DDBMS manages all fragments transparently.
7	Distributed query processing	A distributed query may be executed at several different DP sites. Query optimization is performed transparently by the DDBMS.
8	Distributed transaction processing	A transaction may update data at several different sites, and the transaction is executed transparently.
9	Hardware independence	The system must run on any hardware platform.
10	Operating system independence	The system must run on any operating system platform.
11	Network independence	The system must run on any network platform.
12	Database independence	The system must support any vendor's database product.



Summary

- A distributed database stores logically related data in two or more physically independent sites connected via a computer network
- Distributed processing is the division of logical database processing among two or more network nodes
- The main components of a DDBMS are the transaction processor (TP) and the data processor (DP)
- Current database systems can be classified by the extent to which they support processing and data distribution
- A homogeneous distributed database system integrates only one particular type of DBMS over a computer network
- DDBMS characteristics are best described as a set of transparencies: distribution, transaction, performance, failure, and heterogeneity



Summary

- A transaction is formed by one or more database requests. An undistributed transaction updates or requests data from a single site
- Distributed concurrency control is required in a network of distributed databases
- A distributed DBMS evaluates every data request to find the optimum access path in a distributed database
- The design of a distributed database must consider the fragmentation and replication of data
- A database can be replicated over several different sites on a computer network
- The CAP theorem states that a highly distributed data system has some desirable properties of consistency, availability, and partition tolerance